

Machine Intelligence

12. Planning under Uncertainty: Markov Decision Processes

Álvaro Torralba



AALBORG UNIVERSITET

Fall 2024

In previous chapters of the MI Lecture...

- State-space search **Chapter 9**:
→ The basic algorithms under the hood
- Classical Planning **Chapter 10**:
→ Automated Decision Making with domain-independent techniques. The user only specifies how the environment works and by analyzing the environment (delete-relaxation in our case), we can solve it efficiently!

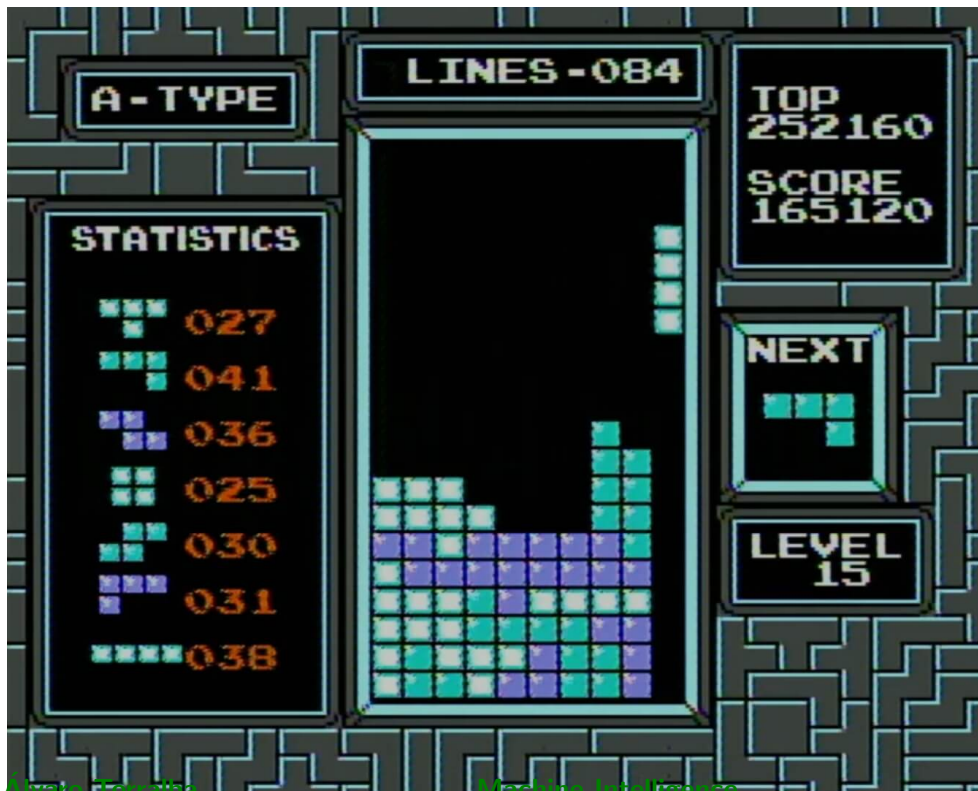
Those techniques give us a **plan to achieve a goal** on environments that are:

- single-agent → we studied multi-agent environments on **Chapter 11**
- deterministic → **(This Chapter)**: non-determinism (MDPs)
- goal-oriented → **(This Chapter)**: reward-oriented (MDPs)
- fully-observable → partially-observable (POMDPs) (not in this lecture)
- discrete
- ...

Decision problems with an unbounded time horizon

Characteristics.

- at each step we are faced with a decision,
- at each step we are given a certain reward (possibly negative) determined by the chosen decision and the state of the world,
- the outcome of a decision may be uncertain (we will assume we know the probabilities associated to each outcome),
- the time horizon of the decision problem is unbounded.



- Non-determinism: Every time we place a piece, the next piece is determined at random (with uniform probability)
- What are the goal states? We cannot characterize “keep alive” as reaching a goal.
- Maximize reward (points) for as much time as possible

Markov Decision Processes

Definition (Markov Decision Process). An **MDP** is a 6-tuple $\Theta = (S, A, r, P, I, S^T)$ where:

- S is a finite set of **states**.
- A is a finite set of **actions**.
- $r : S \times A \times S \mapsto \mathbb{R}$ is the **reward function**.
- $P : S \times A \times S \mapsto [0, 1]$ is the **transition probability function**, representing $P(s' \mid s, a)$, the probability that by applying action a in state s , we end in state s' .
- $I \in S$ is the **initial state**.
- $S^T \subseteq S$ is the set of **terminal states**. The set of terminal states could be empty.

→ We select actions in order to maximize the reward obtained

Similar to the definition of state-space (**c.f. Chapters 9/10**), but:

- Instead of a cost function, we have a reward function.
- Instead of a transition relation, we have a probability distribution over possible outgoing states

Markov Decision Process Example

	1	2	3
1			
2			
3			

- The adventurer wants to take the gold located in (3,1).
- But she should avoid the snake.
- She has a complete knowledge of the environment.
- But paths are slippery and there is a small probability of failing one step and moving to the cell to the right.

To determine what's the best course of action, let's encode the problem as an MDP!

- **Set of states?** One state for each position of the adventurer:

$$S = \{(x, y) \mid x, y \in \{1, 2, 3\}\}$$

Note: this assumes the snake does not move

- **Initial state?** (1, 1)
- **Terminal states?** {(3, 1)}
- **Actions?** {north, east, south, west}

Transition Probabilities in an MDP

The **transition probabilities** $P(S_i | A_{i-1}, S_{i-1})$ tell us what are the possible outcomes of executing each action on each state

- The adventurer can move north, east, south, and west.
- A move succeeds with probability 0.8; otherwise it moves to the location immediately to the right.
- An attempt to move outside the boundaries of the grid results in staying in the current square.

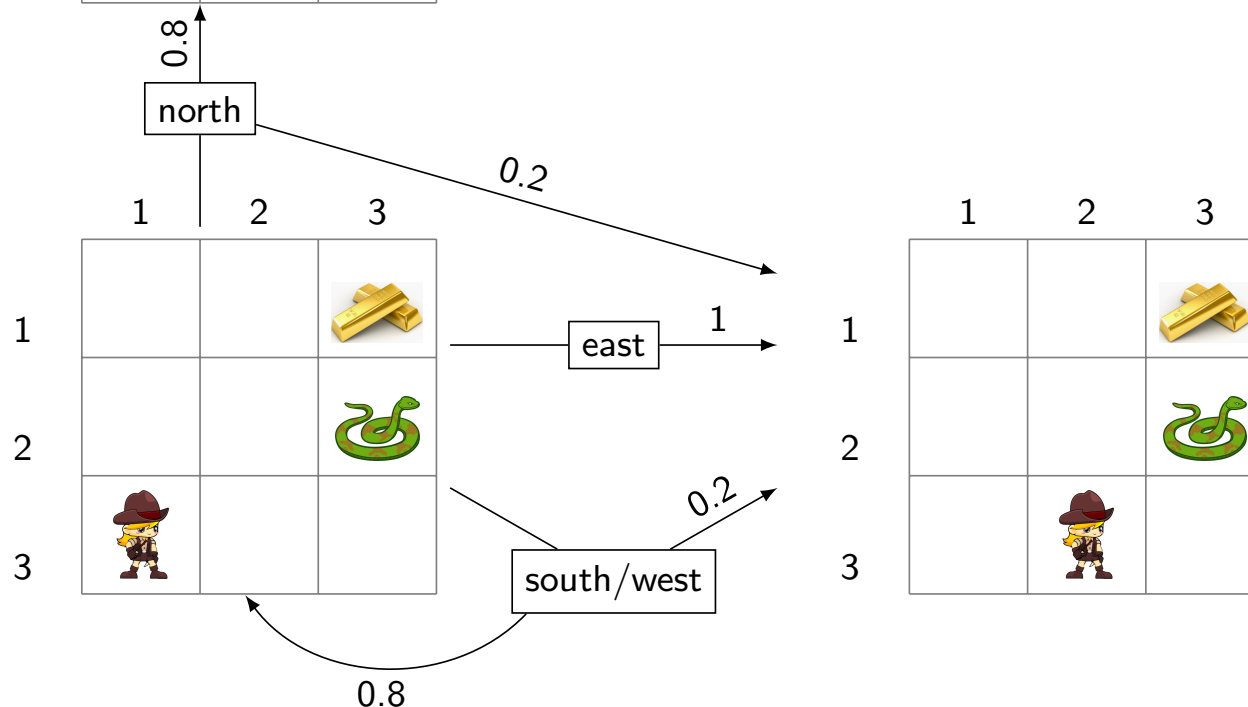
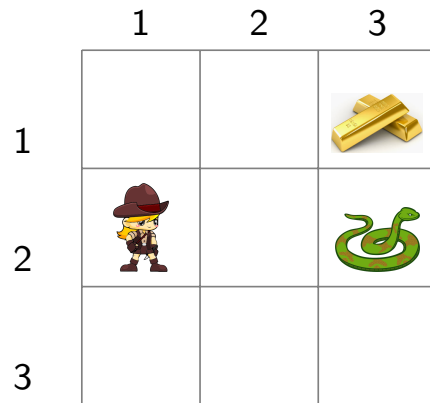
	1	2	3
1			
2			
3			

What are the outcomes of moving north in the initial state?

For example, for the north action we have that $P(S_{i+1} | \text{north}, S_i)$:

(column,row)		S_i								
	(column,row)	(1,1)	(1,2)	(1,3)	(2,1)	(2,2)	(2,3)	(3,1)	(3,2)	(3,3)
S_{i+1}	(1,1)	0.8	0.8	0	0	0	0	0	0	0
	(1,2)	0	0	0.8	0	0	0	0	0	0
	(1,3)	0	0	0	0	0	0	0	0	0
	(2,1)	0.2	0	0	0.8	0.8	0	0	0	0
	(2,2)	0	0.2	0	0	0	0.8	0	0	0
	(2,3)	0	0	0.2	0	0	0	0	0	0
	(3,1)	0	0	0	0.2	0	0	1	0.8	0
	(3,2)	0	0	0	0	0.2	0	0	0.2	0.8
	(3,3)	0	0	0	0	0	0.2	0	0	0.2

Markov Decision Process as a Graph



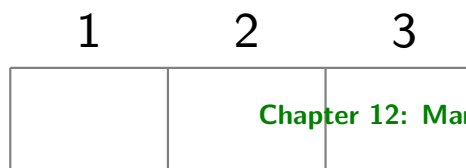
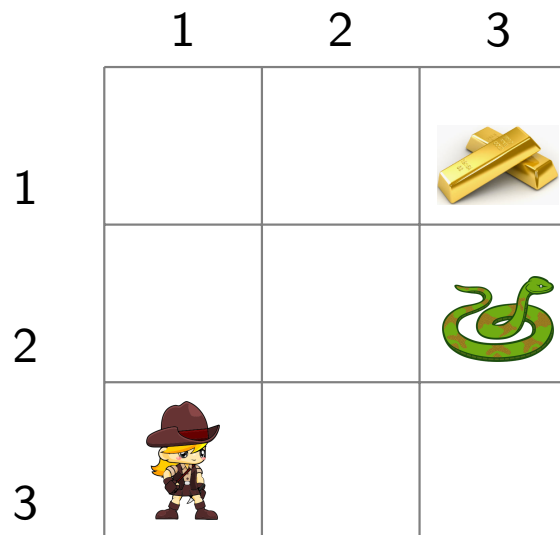
- The state space of an MDP is an hypergraph where each action may have more than one possible outcome
- Similar to MinMax trees from Games, but here we have the probability of each outcome, instead of always picking the worst-case scenario
- The picture in this slide only shows a small portion (the actions applicable in the initial state): to generate the full state-space of the MDP we need to consider all actions in all states.

Rewards

The **reward function** $r(s, a, s')$ specifies “how many points we get” every time we apply an action.

In our example:

- Getting gold: +10
- Meeting snake: -5
- Moving elsewhere: -0.1 (so that shorter paths are preferred)



Markov Assumption

Markov Assumption: given the present, the future does not depend on the past

Where do we see the Markov assumption in Markov Decision Processes?

- Transition probabilities $P(S_i | A_{i-1}, S_{i-1})$
 - The probability that we reach certain state only depends on the present (what state we were and what action we applied)
- Reward function $r(s, a, s')$
 - Again, it does only depend on the present

Example of problem that does not satisfy the Markov assumption?

→ We only get 10 points the first time that we get to the square with the gold

But: We can ensure the markov assumption by incorporating an extra state variable representing whether we already have the gold

Solving MDPs

	1	2	3
1			
2			
3			

Question

What's the plan?

There is no “plan” as a sequence of actions. Since we do not know what the result of our actions will be with full certainty, we need to plan for every possible outcome

Decision policies

A decision policy for MDPs is a function mapping non-terminal states to actions:

$$\pi : S \mapsto A$$

	1	2	3
1	→	→	
2	↑	↑	↑
3	↑	→	↑

In MDPs it suffices to consider this “simple” pure policies. In other settings policies could be more complex:

- A policy could depend on the entire history from the initial state to s
 $(\pi(s_0, a_0, s_1, a_1, \dots, s_i) \mapsto A)$
 → We do not need to take the past into account due to the Markov property
 In other words, if we visit the same state twice, we'd like to do the same action
- A policy could be stochastic (assigning probabilities to actions)
 → We do not need that here (in MDPs there is always some optimal policy that is pure).

Executing a Policy

Given a policy, we can simulate its execution to obtain all possible traces it could have and their corresponding probability and utility.

	1	2	3
1	→	→	
2	↑	↑	↑
3	↑	→	↑

Two example traces:

	1	2	3
1			🔑
2			🐍
3	👤		

$\xrightarrow{N}$

	1	2	3
1			🔑
2	👤		🐍
3			

$\xrightarrow{N}$

	1	2	3
1	👤		🔑
2			🐍
3			

$\xrightarrow{E}$

	1	2	3
1		👤	🔑
2			🐍
3			

$\xrightarrow{E}$

	1	2	3
1			👤
2			🐍
3			

$$\text{Prob} = 0.8 \cdot 0.8 \cdot 1 \cdot 1 = 0.64$$

$$\text{Utility} = -0.1 + -0.1 + -0.1 + 10 = 9.7$$

	1	2	3
1			🔑
2			🐍
3	👤		

$\xrightarrow{N}$

	1	2	3
1			🔑
2			🐍
3		👤	

$\xrightarrow{E}$

	1	2	3
1			🔑
2			🐍
3			👤

$\xrightarrow{N}$

	1	2	3
1			🔑
2			👤
3			

$\xrightarrow{N}$

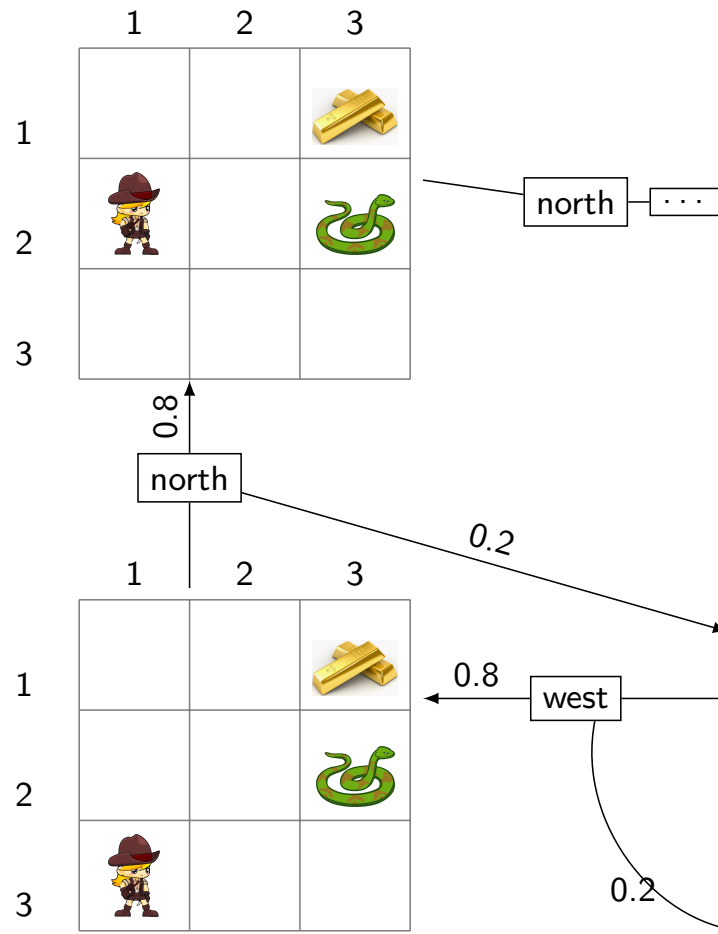
	1	2	3
1			👤
2			🐍
3			

$$\text{Prob} = 0.2 \cdot 1 \cdot 0.8 \cdot 0.8 \approx 0.13$$

$$\text{Utility} = -0.1 + -0.1 + -5 + 10 = 4.8$$

Markov Chains

The **Markov chain** associated to a policy π of an MDP is the (possibly infinite) set of traces with their corresponding probabilities.



Possible executions:

- $(1, 3) \xrightarrow{N} (1, 2) \xrightarrow{N} \dots$
- $(1, 3) \xrightarrow{N} (2, 3) \xrightarrow{W} (1, 3) \xrightarrow{N} (1, 2) \xrightarrow{N} \dots$
- $(1, 3) \xrightarrow{N} (2, 3) \xrightarrow{W} (1, 3) \xrightarrow{N} (2, 3) \xrightarrow{W} (1, 3) \xrightarrow{N} (1, 2) \xrightarrow{N} \dots$
- etc.

We could define the **expected utility** as the average reward obtained from each trace (weighted by its probability), but there is a problem (see next).

Evaluating strategies

Imagine that we have the following reward function and **there is no terminal state** and no uncertainty on the result of an action:

	1	2	3
1	-0.1	-0.1	10
2	-0.1	-0.1	-5
3	-0.1	-0.1	-0.1

Question!

Which strategy is best according to expected utility with unbounded horizon?

(A): $\pi_A =$

	1	2	3
1	→	→	←
2	↑	↑	↑
3	↑	→	↑

(B): $\pi_B =$

	1	2	3
1	→	→	↓
2	↑	↑	↓
3	↑	←	←

$$EU(\pi_A) = -0.1 + -0.1 + -0.1 + 10 + -0.1 + 10 + -0.1 + 10 + \dots = \infty$$

$$EU(\pi_B) = -0.1 + -0.1 + -0.1 + 10 + -5 + -0.1 + -0.1 + -0.1 + \dots = \infty$$

Beyond Expected Sum of Rewards

Problem: Sum of rewards over an infinite horizon is often infinite, no matter how much you wait to start gaining reward

Two solutions:

- **Finite Horizon**
- **Discounted Rewards**

The utility of an unbounded sequence: fixed horizon

Finite-Horizon MDP

Consider only the rewards obtained in the first k states:

$$U(s_0, s_1, s_2, \dots) = R(s_0) + R(s_1) + \dots + R(s_k) < \infty.$$

But how do we choose k ?

- For $k = 0$ we only care about the immediate reward, hence we pursue a very greedy strategy.
- For, say, $k = 5$, we only care about maximizing the reward the next 5 time steps (but we may end in a really bad state for the future)

Finite horizon MDPs = Live your life as if there were not tomorrow

Question!

Do finite-horizon MDPs satisfy the Markov property?

(A): Yes

(B): No

Yes, but the horizon should be part of the state (how much time alive we have).

Discounted rewards

Compare reward sequences

r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8	r_9	r_{10}	r_{11}	...
-0.1	10.0	-0.1	10.0	-0.1	10.0	-0.1	10.0	-0.1	10.0	-0.1	...
-0.1	-0.1	-0.1	-0.1	10.0	-0.1	-0.1	-0.1	-0.1	10.0	-0.1	...

We prefer first sequence, because we receive **“more reward sooner”**!

This can be captured by **discounted rewards**:

$$U(s_0, s_1, s_2, s_3, \dots) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \gamma^3 R(s_3) + \dots,$$

where $0 \leq \gamma \leq 1$.

The utility of an unbounded sequence: discounted rewards

Weigh rewards in the immediate future higher than rewards in the distant future:

$$U(s_0, s_1, s_2, \dots) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots,$$

where $0 \leq \gamma \leq 1$. Possible interpretations of the **discounting factor** γ :

- In economics, γ may be thought of as inflation or an interest rate.
- The decision process may terminate with probability $(1 - \gamma)$ at any point in time, e.g. the robot breaking down.

Thanks to increasing the exponent in gamma after every step, the sum of rewards is always finite!

With $\gamma < 1$ we have:

$$U(s_0, s_1, s_2, \dots) = \sum_{i=0}^{\infty} \gamma^i R(s_i) \leq \sum_{i=0}^{\infty} \gamma^i \max R = \frac{\max R}{1 - \gamma} < \infty.$$

- For $\gamma = 0$ we have a greedy strategy.
- For $\gamma = 1$ we have normal additive rewards.

→ So, **expected discounted reward** is a good metric to compare policies → Typically, we want values $\gamma < 1$ but $\gamma \approx 1$, such as 0.99

In summary What's the Objective?

- **Immediate reward** at time t : r_t (obviously inadequate)

- **Cumulative reward** at time t :

$$r_t + r_{t+1} + r_{t+2} + \dots = \sum_{i=0}^{\infty} r_{t+i} = \sum_{i \geq 0} r_{t+i}$$

→ Diverges if a cycle in the policy envelope yields positive reward.

- **Cumulative finite-horizon reward** at time t :

$$r_t + r_{t+1} + \dots + r_{t+N} = \sum_{i=0}^N r_{t+i}$$

→ Reward beyond N steps is ignored.

- **Cumulative discounted reward** at time t : **used in discounted MDPs!**

$$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{i \geq 0} \gamma^i r_{t+i}$$

→ Guarantees convergence (sum is always finite). Importance of events decreases exponentially over time.

The Policy-Value Function

The actions may be non-deterministic so a strategy may only take you to a state with a certain probability.



Reminder: Cumulative discounted reward

$$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{i \geq 0} \gamma^i r_{t+i}$$

with rewards r_{t+i} from a state sequence arising from π and T .

→ **The state value is the expectation over such state sequences:**

Definition. Given a discounted MDP $(S, A, r, P, I, S^T, \gamma)$, $s \in S$, and a policy π closed in s , the **value of s under π** is

$$\begin{aligned} V^\pi(s) &= \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots] \\ &= \mathbb{E}[\sum_{i \geq 0} \gamma^i r_{t+i}] \end{aligned}$$

where $r_{t+i} = R(s_{t+i})$ given a state sequence $s_t, s_{t+1}, \dots$ induced by π and T starting from $s = s_t$, and the expectation is over these sequences.

Optimal Value Function and Optimal Policy

Definition (V^*). Given a discounted MDP $(S, A, r, P, I, S^T, \gamma)$, the **optimal value function** V^* is defined as $V^*(s) = \max_{\pi} V^{\pi}(s)$.

Theorem (V^* is Finite). Given a discounted MDP $(S, A, r, P, I, S^T, \gamma)$, for every $s \in S$ it holds that $V^*(s) \in \mathbb{R}$.

→ Thanks to $\gamma < 1$ which prevents divergence.

Definition (Optimal Policy). Given a discounted MDP $(S, A, r, P, I, S^T, \gamma)$, a policy π is **optimal** if $V^{\pi} = V^*$. We sometimes denote π^* to indicate that a policy is optimal.

The Bellman Equation

Theorem (The Bellman Equation). Given a discounted MDP $(S, A, r, P, I, S^T, \gamma)$, for every $s \in S$ it holds that

$$\mathbf{V}^*(s) = \mathbf{R}(s) + \max_{\mathbf{a} \in \mathbf{A}} \gamma \sum_{s'} \mathbf{P}(s'|s, \mathbf{a}) \mathbf{V}^*(s')$$

→ Optimal state value = immediate reward + best action's (discounted) expected successor state value.

Uses of the Bellman equation:

- **Bellman state update:** given value function V , set

$$V(s) \leftarrow R(s) + \max_{a \in A} \gamma \sum_{s'} P(s'|s, a) V(s')$$

→ Such updates (starting from any V) converge to V^* .

- Compute an optimal policy! We have:

$$\pi^*(s) = \arg \max_{a \in A} \gamma \sum_{s'} P(s'|s, a) V^*(s')$$

- **Bellman operator:** given value function V , the function $\mathcal{B}V$ is defined by

$$\mathcal{B}V(s) = R(s) + \max_{a \in A} \gamma \sum_{s'} P(s'|s, a) V(s') \text{ for all } s$$

→ Applies Bellman state update to all states in parallel.

Value Iteration (VI)

Iterate Bellman operator:

```
function ValueIteration(discounted MDP,  $\epsilon$ ) returns  $\epsilon$ -approximation of  $V^*$ 
  for each  $s \in S$  do  $V^0[s] \leftarrow R(s)$ 
   $i \leftarrow 0$ 
  repeat
     $i \leftarrow i + 1$ 
    for each  $s \in S$  do
       $V^i[s] \leftarrow R(s) + \max_{a \in A} \gamma \sum_{s'} P(s'|s, a) V^{i-1}[s']$ 
     $\delta \leftarrow \max_{s \in S} |V^i(s) - V^{i-1}(s)|$ 
  until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
```

- Store current and next value functions in arrays V , V' .
- Initialization of V^0 can be arbitrary in principle (in the next slide we use 0).
- One can show that, upon termination with $\delta < \epsilon(1 - \gamma)/\gamma$, we have $|V^*(s) - V[s]| < \epsilon$ for all s .
- Same algorithm applies (with adaptations to Bellman equation) to all classes of MDPs, including in particular SSPs.

Value iteration: an example

	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

Initialization V^0

	1	2	3
1	-0.1	-0.1	10
2	-0.1	-0.1	-5
3	-0.1	-0.1	-0.1

First iteration V^1

	1	2	3
1	-0.19	8.9	10
2	-0.19	-1.07	1.3
3	-0.19	-0.19	-0.19

Second iteration V^2

$$V^1(1, 2) = R(1, 2) + \gamma \max \left\{ \sum_s P(s | \text{north}, (1, 2)) V^0(s), \dots, \sum_s P(s | \text{west}, (1, 2)) V^0(s) \right\}$$

$$= -0.1 + 0.9 \cdot \max\{0, 0, 0, 0\} = -0.1.$$

$$V^1(2, 2) = R(2, 2) + \gamma \max \left\{ \sum_s P(s | \text{north}, (2, 2)) V^0(s), \dots, \sum_s P(s | \text{west}, (2, 2)) V^0(s) \right\}$$

$$= -0.1 + 0.9 \cdot \max\{0, 0, 0, 0\} = -0.1.$$

$$V^2(2, 1) = -0.1 + 0.9 \cdot \max\{0.8 \cdot -0.1 + 0.2 \cdot 10$$

Value iteration: the impact of the discounting factor

	1	2	3
1	7.91	8.9	10
2	6.82	6.79	2.62
3	5.83	5.66	4.85

The utility function for $\gamma = 0.9$
(51 iterations)

	1	2	3
1	→	→	
2	↑	↑	↑
3	↑	↑	←

The optimal strategy for $\gamma = 0.9$

	1	2	3
1	-0.01	0.9	10
2	-0.1	-0.11	-4.29
3	-0.11	-0.11	-0.11

The utility function for $\gamma = 0.1$ (17 iterations)

	1	2	3
1	→	→	
2	↑	↑	↑
3	↑	←	←

The optimal strategy for $\gamma = 0.1$

Value iteration: convergence

So the algorithm converges for this particular example, but does this hold in general?

Yes, it can be proven that there is only one “true” utility function and value iteration is guaranteed to converge to this utility function. Moreover,

$$m = \frac{\log(\epsilon(1 - \gamma)/2R_{\max})}{\log(\gamma)}$$

is an upper bound on the number of iterations required to achieve an error less than ϵ .

Value Iteration: Discussion

Is VI a practical algorithm for solving MDPs? Not really:

- It requires to loop over all states in Θ .
- Multiple times, until convergence.

→ **This can work only if the MDP state space Θ is small.**

That said: (compared to classical planning)

- Finding optimal policies for MDPs is very hard (one aspect: solution size, policy vs. path to goal).
- Scalability is bad, often only “small” (millions of states) MDPs are feasible, even for algorithms much more clever than VI.
- VI is very generally applicable, also beyond dMDPs as defined here.

→ **Nevertheless, there are algorithms that can find optimal policies without looking at all states in an MDP. See next slide.**

Beyond Value Iteration

There are other algorithms beyond value iteration:

- Policy Iteration: Instead of converging to the optimal value function V^* and then extracting the optimal policy, try to converge by changing the policy in each iteration.
- Heuristic Search: Value Iteration enumerates the entire state space before starting to solve the algorithm (like naive implementations of Dijkstra's algorithm). As in A^* , we can reduce the portion of the state-space that we consider by using admissible heuristics.
- Online decision making: There are also algorithms that, instead of computing the entire policy in advance, try to decide what action to perform in the current state.

Summary

In general, in a **Markov decision process**:

- the world is *fully observable*, i.e., the agent can observe the true state of the world at any point in time,
- the uncertainty in the system is a result of the consequences of the actions being non-deterministic (when performing an action we make a state transition with a certain probability, but independent of the time step), and
- for each decision we get a reward (which may be negative) that may depend on the current world state but is independent of the time step.

Value Iteration can be used to find the optimal policy of any MDP.

Reading

- *Chapter 9 Planning with Uncertainty* from the book "Artificial Intelligence: Foundations of Computational Agents" (2nd edition); in particular subchapters:
 - 9.1. Preferences and Utility
 - 9.2. One-Off Decisions
 - 9.3. Sequential Decisions
 - 9.4. The value of information and control
 - 9.5. Decision Processes

Policy iteration

Instead of updating the utility function, make an initial guess at the optimal policy and perform an iterative refinement of this guess:

The **updating function**:

$$\pi_{i+1}(s) := \arg \max_a \sum_{s' \in S} P(s' | a, s) U_{\pi_i}(s').$$

The **evaluation function**:

$$U_{\pi_i}(s) = R(s) + \gamma \sum_{s' \in S} P(s' | \pi_i(s), s) U_{\pi_i}(s'),$$

which defines a system of linear equalities; the solution is U_{π_i} .

Policy iteration: the algorithm

- ① Let π_0 be an initial randomly chosen policy.
- ② Set $i := 0$.
- ③ **Repeat**
 - ① Find the utility function U_{π_i} corresponding to the policy π_i [Policy evaluation].
 - ② Let $i := i + 1$.
 - ③ **For** each $s \in S$

$$\pi_i(s) := \arg \max_a \sum_{s' \in S} P(s' | a, s) U_{\pi_{i-1}}(s') [\text{Policy updating}].$$

- ④ **Until** $\pi_i = \pi_{i-1}$

Policy evaluation can be performed both exactly ($O(n^3)$) or approximately using a simplified version of value iteration.